

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	Date:
Day:	Semester:

Lab 6: K-Means Clustering from Scratch

Lab Title	CLO	BT	PLO	Weightage
K-Means Clustering From Scratch				

Name	Roll Number	Report /10	Viva /5	Total /15

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab students will be able to:

- Implement the K-Means clustering algorithm from scratch.
- Understand how different distance metrics affect clustering.
- Compare clustering results on separable vs overlapping datasets.
- Visualize cluster decision boundaries.

2 Datasets

In this lab we will experiment with two datasets.

2.1 Dataset 1: Iris Dataset

The Iris dataset contains 150 samples.

For visualization purposes we only use two features:

- Sepal Length
- Petal Length

2.2 Dataset 2: Synthetic Dataset

We will also generate a synthetic dataset using:

```
sklearn.datasets.make_blobs
```

Two versions will be created:

- Well-separated clusters
- Overlapping clusters

This allows us to observe how K-Means behaves when clusters are not clearly separable.

3 Distance Metrics

In this lab you will experiment with three distance metrics.

3.1 Euclidean Distance

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Produces circular cluster boundaries.

3.2 Manhattan Distance

$$d(x, y) = \sum |x_i - y_i|$$

Produces diamond-shaped cluster boundaries.

3.3 Cosine Distance

Cosine similarity measures the angle between two vectors.

$$\text{similarity} = \frac{x \cdot y}{||x|| ||y||}$$

Cosine distance:

$$d(x, y) = 1 - \text{similarity}$$

This metric is commonly used in:

- Text clustering
- Document similarity
- Embedding spaces

4 Part A: Distance Function (3 Marks)

Implement the function:

```
_distance(x1, x2)
```

The function must support the following metrics:

- Euclidean
- Manhattan
- Cosine

Use NumPy operations only.

5 Part B: Cluster Assignment (4 Marks)

Implement:

```
assign_clusters(X, centroids)
```

Steps:

- Compute the distance from each point to every centroid
- Assign the point to the closest centroid
- Return cluster labels

6 Part C: Update Centroids (4 Marks)

Implement:

```
update_centroids(X, labels)
```

Steps:

- For each cluster compute the mean of all assigned points
- Return updated centroid coordinates

7 Part D: Full K-Means Algorithm (4 Marks)

Implement:

```
fit(X)  
predict(X)
```

Algorithm:

1. Initialize K centroids randomly
2. Assign points to nearest centroid
3. Update centroid locations
4. Repeat until centroids converge or max iterations reached

8 Part E: Boundary Visualization (Bonus)

Create a function:

```
plot_decision_boundaries(model, X)
```

Steps:

- Create a mesh grid over the feature space
- Predict cluster label for each grid point
- Plot colored decision regions

Run the visualization for:

- Euclidean distance
- Manhattan distance
- Cosine distance

9 Part F: Experimental Comparison

Run K-Means on both datasets.

Dataset	Distance Metric	Observations
Iris	Euclidean	
Iris	Manhattan	
Iris	Cosine	
Separable Blobs	Euclidean	
Overlapping Blobs	Euclidean	

Discuss:

- How do distance metrics change cluster shapes?
- Which metric performs best for overlapping clusters?
- Does K-Means always converge to the same solution?

10 Cluster Quality: Within Cluster Sum of Squares

K-Means attempts to minimize the following objective function:

$$J = \sum_{i=1}^n ||x_i - \mu_{c_i}||^2$$

Where:

- x_i is a data point
- μ_{c_i} is the centroid of the cluster assigned to x_i

This value is often called:

- Within Cluster Sum of Squares (WCSS) or Sum of Squared Errors (SSE)
- Inertia

Lower values indicate tighter clusters.

11 Choosing the Number of Clusters: Elbow Method

The number of clusters K is usually unknown in real applications.

The **Elbow Method** helps estimate a reasonable value for K .

Procedure:

1. Run K-Means for different values of K
2. Compute the inertia for each K
3. Plot inertia vs K

Typically the curve looks like:

High decrease $\rightarrow$ elbow $\rightarrow$ slow decrease

The “elbow” point suggests the optimal number of clusters.

12 Part F: Compute Inertia (3 Marks)

Implement the function:

```
compute_inertia(X, labels, centroids)
```

Steps:

- For every point compute the squared distance to its centroid
- Sum the squared distances
- Return the total inertia

13 Part G: Elbow Method (Bonus)

Create a function:

```
plot_elbow_curve(X, k_values)
```

Steps:

- Train K-Means for multiple values of K
- Compute inertia for each value
- Plot inertia vs K

Test with:

```
k_values = range(1,10)
```

Which value of K appears optimal for the dataset?